

# Week 1 Documentation – Microservices Infrastructure Setup

## 1. Objective of Week 1 (Interview Perspective)

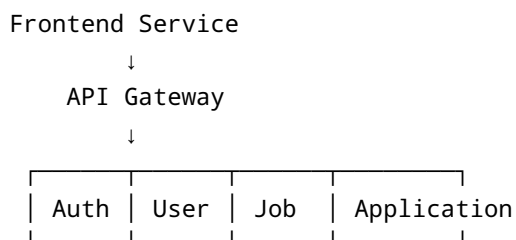
The objective of Week 1 was to **intentionally delay business logic** and instead establish a **production-grade microservices foundation**. In real-world backend systems, poor early infrastructure decisions lead to long-term scalability, deployment, and reliability issues.

The focus of this phase was to answer three critical questions upfront: 1. Can services be developed and deployed independently? 2. Can they reliably discover and communicate with each other? 3. Is the system structured in a way that scales with team size and feature complexity?

By solving these problems first, all subsequent feature development becomes safer, faster, and more predictable.

---

## 2. High-Level Architecture



### Architectural Principles Followed

- **Single Responsibility per service**
- **API Gateway as the only external entry point**
- **Loose coupling via HTTP-based communication**
- **Independent deployability**

---

## 3. Folder Structure (Monorepo with Microservices)

```
job-portal/  
├── api-gateway/  
│   ├── app/  
│   └── main.py
```

```
|   ├── Dockerfile
|   └── pyproject.toml
├── auth-service/
├── user-service/
├── job-service/
├── application-service/
├── frontend-service/
├── docker-compose.yml
└── README.md
```

### Why This Structure?

- **Clear service boundaries**
- Enables independent scaling and deployments
- Simplifies ownership in team environments
- Avoids shared code that leads to tight coupling

## 4. Dockerization Strategy (Why Docker Per Service)

Each service was containerized using its **own Dockerfile**, rather than sharing a base image or runtime.

### Interview Rationale

- In real systems, services evolve at different speeds
- Independent Dockerfiles allow independent upgrades
- Prevents dependency conflicts across services
- Enables horizontal scaling and isolated failures

This approach mirrors how services are built and deployed in Kubernetes-based environments.

### Example Dockerfile (FastAPI Service)

```
FROM python:3.11-slim

WORKDIR /app

COPY pyproject.toml uv.lock ./
RUN pip install uv && uv sync --frozen

COPY app ./app

CMD ["uv", "run", "uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Key interview point:

*Each service owns its runtime, dependencies, and startup lifecycle.*

---

## 5. docker-compose Orchestration

### Purpose

`docker-compose` is used to: - Start all services with a single command - Create a shared Docker network - Enable service discovery via DNS

### Key Concept: Docker DNS

Each service can be accessed using its **service name** as hostname.

Example:

```
http://job-service:8000
```

This eliminates hardcoded IPs and enables portability.

---

## 6. Health Check Endpoints (Operational Readiness)

Each service exposes a `/health` endpoint to indicate basic liveness.

### Example (FastAPI)

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

### Why This Matters in Production

- Used by container orchestrators for restarts
- Required by load balancers
- First signal during outages
- Helps distinguish infra issues from application bugs

Interview framing:

*Health checks are the minimal contract between a service and the platform running it.*

---

## 7. API Gateway Validation (Critical Proof Point)

The most important validation in Week 1 was confirming **real inter-service communication**.

### Test Performed

A request was executed **from inside the API Gateway container**:

```
docker-compose exec api-gateway sh
curl http://job-service:8000/health
```

### Result

```
{"status": "ok"}
```

### Why This Is Significant

This confirms: - Docker DNS-based service discovery - Correct network isolation - No usage of localhost or hardcoded IPs - Gateway can act as a real entry point

Interview-ready takeaway:

*If this step works, the entire microservices networking layer is correctly configured.*

---

## 8. Frontend as an Independent Service

Although no frontend logic is implemented, a **frontend service container** exists.

### Why This Was Done

- Reinforces backend–frontend separation
  - Simulates real production architecture
  - Enables learning of CORS, gateway routing, and API contracts later
- 

## 9. Benefits of This Approach (Interview Value)

### Technical Benefits

- Early architectural validation
- Clear separation of concerns

- Safe parallel development
- Reduced production risk

## Interview & Career Benefits

This setup allows confident discussion around: - Microservices trade-offs - API Gateway responsibilities - Docker networking and service discovery - Infra-first development mindset

Key interview signal:

*Demonstrates system thinking beyond CRUD-level development.*

---

## 10. Known Limitations / Shortcomings (Intentional)

| Limitation                 | Reason                               |
|----------------------------|--------------------------------------|
| No database                | Deferred to avoid premature coupling |
| No auth logic              | Focus on infra first                 |
| No service-to-service auth | Will be added later (JWT/mTLS)       |
| No observability           | Logging & tracing planned later      |

These are **conscious trade-offs**, not oversights.

---

## 11. How This Evolves in Next Weeks

Week 2 will introduce: - API Gateway routing logic - Downstream HTTP clients - Error handling & timeouts

Week 3-4 will add: - Databases per service - Authentication & authorization - End-to-end request flow

---

## 12. Executive Summary (Interview Ready)

"I began by designing and validating a production-style microservices infrastructure using Docker and an API Gateway. I ensured service isolation, health monitoring, and inter-service communication before implementing any business logic, which significantly reduced architectural risk later in development."

**Status:**  Week 1 completed successfully